

University of Jordan
School of Engineering
Department of Electrical Engineering

Three-Phase Inverter Design and Implementation Report

ESP32-Based SPWM Control with IR2101 Gate Drivers

Course: Microcontroller Applications Lab

Prepared by: Ghaith Wael

Student ID: 0226381

Instructor: Prof. Jamal Rahhal

Submitted on: May 16, 2026

Abstract

This report presents a 24 V ESP32-based three-phase inverter using SPWM control. The inverter generates a variable-frequency three-phase output from 1 Hz to 50 Hz using a potentiometer. Three IR2101 gate drivers drive a six-MOSFET bridge built with IRFZ44 MOSFETs, while the ESP32 MCPWM module provides complementary PWM signals with dead time to reduce shoot-through risk. The main component values were selected using datasheet-based calculations, and the operation was verified using oscilloscope measurements.

Contents

1	Introduction	2
2	System Overview	3
2.1	System Block Diagram	3
2.2	Main System Specifications	3
3	Hardware Design and Circuit Operation	4
3.1	Complete Inverter Schematic	4
3.2	ESP32 to IR2101 Signal Connections	5
3.3	IR2101 Gate Driver Stage	6
3.4	Bootstrap Circuit Operation	6
3.5	Three-Phase MOSFET Bridge	7
3.6	Shoot-Through Protection	7
4	Control Logic and ESP32 Waveforms	8
4.1	Software Flowchart	8
4.2	ESP32 MCPWM Module	10
4.3	SPWM Duty-Cycle Generation	11
4.4	Six ESP32 PWM Command Signals	12
4.5	Complementary Switching and Dead Time	12
4.6	Potentiometer-Based Frequency Control	13
4.7	Code Implementation and Availability	14
5	Component Selection and Calculations	17
5.1	Gate Resistor Selection	17
5.2	Bootstrap Capacitor Selection	18
5.3	Dead-Time Selection	19
5.4	Wire Selection	19
5.5	Heatsink Selection and Thermal Check	20
6	Experimental Results and Oscilloscope Measurements	21
6.1	ESP32 PWM Output Measurements	21
6.2	Three-Phase Inverter Output Measurements	21
6.3	Measurement Discussion	23
7	Discussion and Conclusion	23
8	References	23

1 Introduction

A three-phase inverter converts a DC input into a controlled three-phase output and is commonly used in motor-drive applications. In this project, a 24 V DC supply is converted into a variable-frequency three-phase output using an ESP32, IR2101 gate drivers, and six IRFZ44 MOSFETs.

The ESP32 generates SPWM signals for the three inverter phases. The three duty cycles are based on sinusoidal references shifted by 120° , and the output frequency is adjusted from 1 Hz to 50 Hz using a potentiometer connected to the ESP32 ADC input.

Since the ESP32 cannot directly drive the MOSFET gates, especially the high-side switches, IR2101 gate drivers are used. Each driver controls one half-bridge and uses a bootstrap circuit for high-side operation. Complementary PWM with dead time is generated using the ESP32 MCPWM module to prevent the high-side and low-side MOSFETs in the same leg from turning on at the same time.

This report describes the circuit design, control method, component selection calculations, and experimental verification of the inverter using oscilloscope measurements.

2 System Overview

2.1 System Block Diagram

The inverter system is divided into five main stages: the potentiometer input, the ESP32 controller, the IR2101 gate-driver stage, the three-phase MOSFET bridge, and the three-phase output/load. The potentiometer sets the required output frequency, while the ESP32 generates the SPWM control signals. These signals are then passed to the IR2101 gate drivers, which provide the required gate-drive voltage and current for the MOSFET bridge.

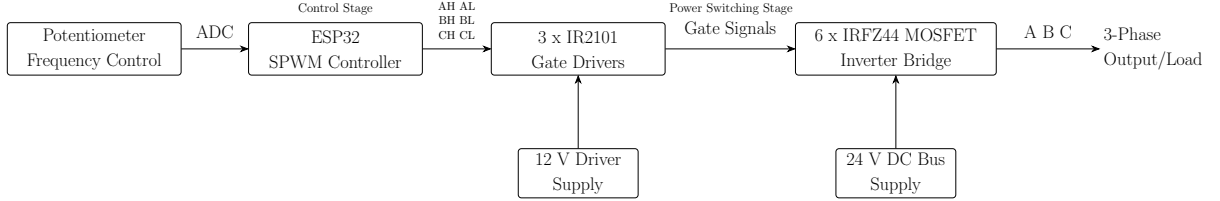


Figure 1: Overall System Block Diagram.

2.2 Main System Specifications

Table 1 summarizes the main specifications of the implemented three-phase inverter.

Table 1: Main system specifications.

Parameter	Value
DC input voltage	24 V
Controller	ESP32
Gate driver	IR2101
MOSFET	IRFZ44
Inverter topology	Three-phase, six-switch bridge
Gate-drive voltage	12 V
PWM carrier frequency	10 kHz
Output frequency range	1 Hz to 50 Hz
Control method	Sinusoidal PWM (SPWM)
Frequency control	Potentiometer through ESP32 ADC
Gate resistor	75 Ω
Bootstrap capacitor	470 nF
Heatsink thermal resistance	20°C/W

3 Hardware Design and Circuit Operation

3.1 Complete Inverter Schematic

Figure 2 shows the complete schematic of the implemented three-phase inverter. The circuit consists of an ESP32 controller, three IR2101 high-side and low-side gate drivers, six IRFZ44 MOSFETs arranged as a three-phase bridge, and bootstrap circuits for the high-side drivers. The inverter is supplied from a 24 V DC bus, while the IR2101 drivers use a 12 V gate-drive supply.

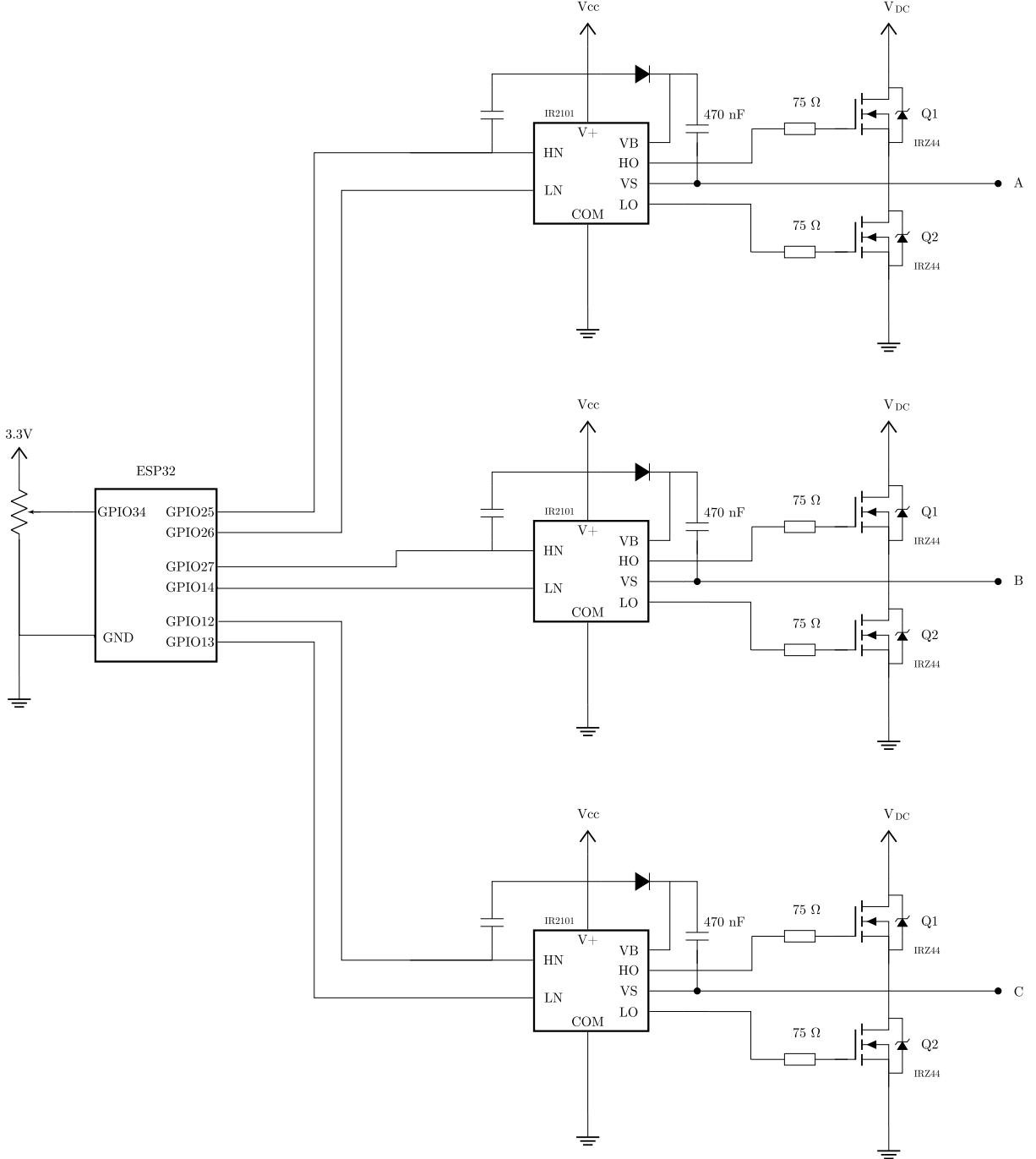


Figure 2: Complete schematic of the ESP32-based three-phase inverter.

3.2 ESP32 to IR2101 Signal Connections

Figure 3 shows the control-signal connections between the ESP32 and the three IR2101 gate drivers. Each IR2101 controls one inverter phase leg. The high-side input pin, HIN, receives the high-side PWM command, while the low-side input pin, LIN, receives the complementary low-side command.

Table 2: ESP32 to IR2101 signal mapping.

ESP32 Pin	Signal	IR2101 Input	Function
GPIO25	AH	U1 HIN	Phase A high-side command
GPIO26	AL	U1 LIN	Phase A low-side command
GPIO27	BH	U2 HIN	Phase B high-side command
GPIO14	BL	U2 LIN	Phase B low-side command
GPIO12	CH	U3 HIN	Phase C high-side command
GPIO13	CL	U3 LIN	Phase C low-side command
GPIO34	POT	ESP32 ADC	Frequency control input

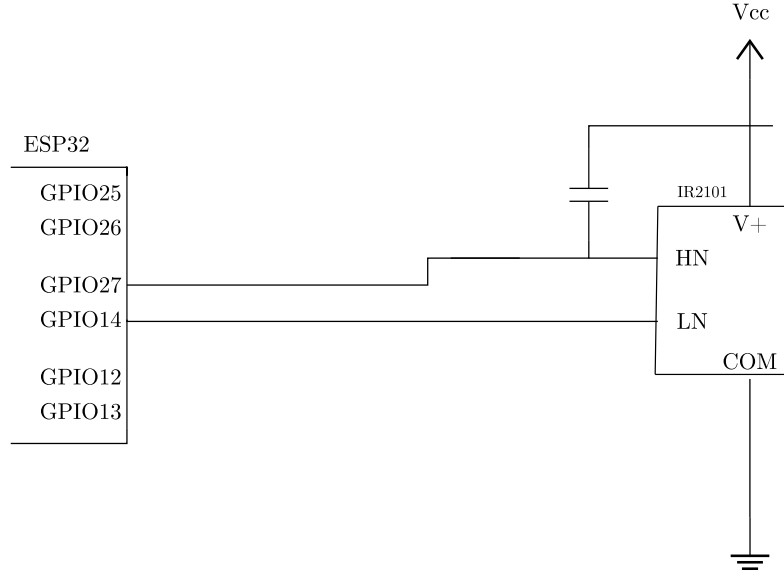


Figure 3: Zoomed schematic of the ESP32 control connections to one of the IR2101 gate drivers.

The ESP32 generates six PWM command signals: AH, AL, BH, BL, CH, and CL. These signals are connected to the HIN and LIN inputs of the three IR2101 drivers, allowing the ESP32 to control the high-side and low-side MOSFETs of each inverter leg.

3.3 IR2101 Gate Driver Stage

Each inverter phase leg is driven using one IR2101 high-side and low-side gate driver. The IR2101 is used because the ESP32 cannot directly drive the MOSFET gates, especially the high-side MOSFET whose source voltage moves with the switching node. The driver increases the gate-drive level to approximately 12 V and provides the required charging and discharging current for the MOSFET gates.

For each phase, the ESP32 high-side command is connected to HIN, while the low-side command is connected to LIN. The corresponding driver outputs are HO and LO, where HO drives the high-side MOSFET gate and LO drives the low-side MOSFET gate. The low-side driver is referenced to COM, while the high-side driver uses the floating supply formed by VB and VS.

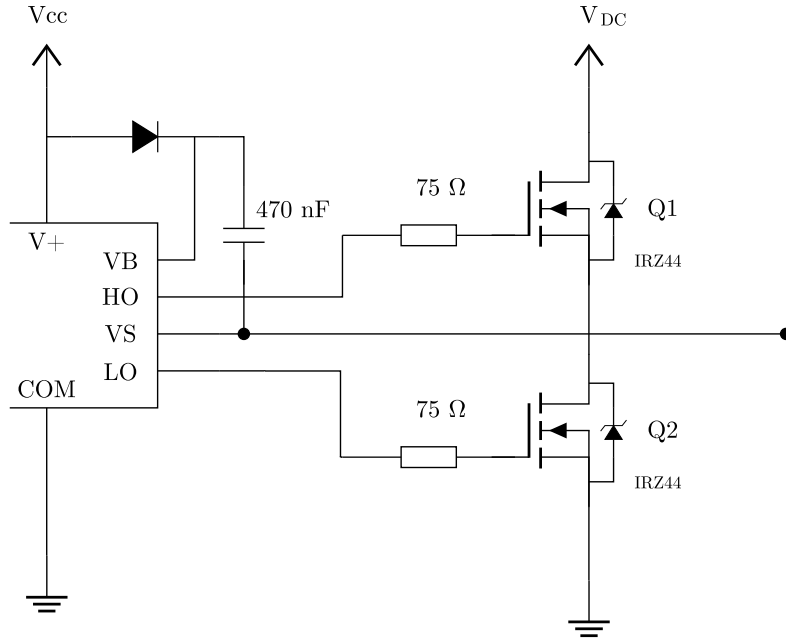


Figure 4: Zoomed schematic of one IR2101 half-bridge gate driver stage.

The same driver circuit is repeated three times for phases A, B, and C. This allows the ESP32 low-voltage PWM signals to control the six MOSFETs of the inverter safely and with proper gate-drive voltage.

3.4 Bootstrap Circuit Operation

The IR2101 uses a bootstrap circuit to drive the high-side MOSFET. This is required because the source of the high-side MOSFET is connected to the switching node, so it moves between ground and the DC bus voltage. To turn the high-side N-channel MOSFET ON, its gate must be driven above its source voltage.

The bootstrap circuit shown in Figure 4 consists of a diode and a capacitor connected between VB and VS. When the low-side MOSFET is ON, the switching node VS is pulled close to ground, so the bootstrap capacitor charges from the 12 V driver supply through the bootstrap diode. When the high-side MOSFET turns ON, the switching node rises, the diode becomes reverse-biased, and the charged capacitor supplies the floating high-side driver.

3.5 Three-Phase MOSFET Bridge

The power stage of the inverter is formed by six IRFZ44 N-channel MOSFETs arranged as a standard three-phase bridge. Each phase leg contains one high-side MOSFET and one low-side MOSFET. Phase A is formed by Q_1 and Q_2 , phase B by Q_3 and Q_4 , and phase C by Q_5 and Q_6 .

The output of each phase is taken from the midpoint of its half-bridge. When the high-side MOSFET is switched ON, the phase node is connected to the positive DC bus. When the low-side MOSFET is switched ON, the phase node is connected to ground. By switching the three legs using SPWM signals shifted by 120° , a three-phase output is produced from the 24 V DC input.

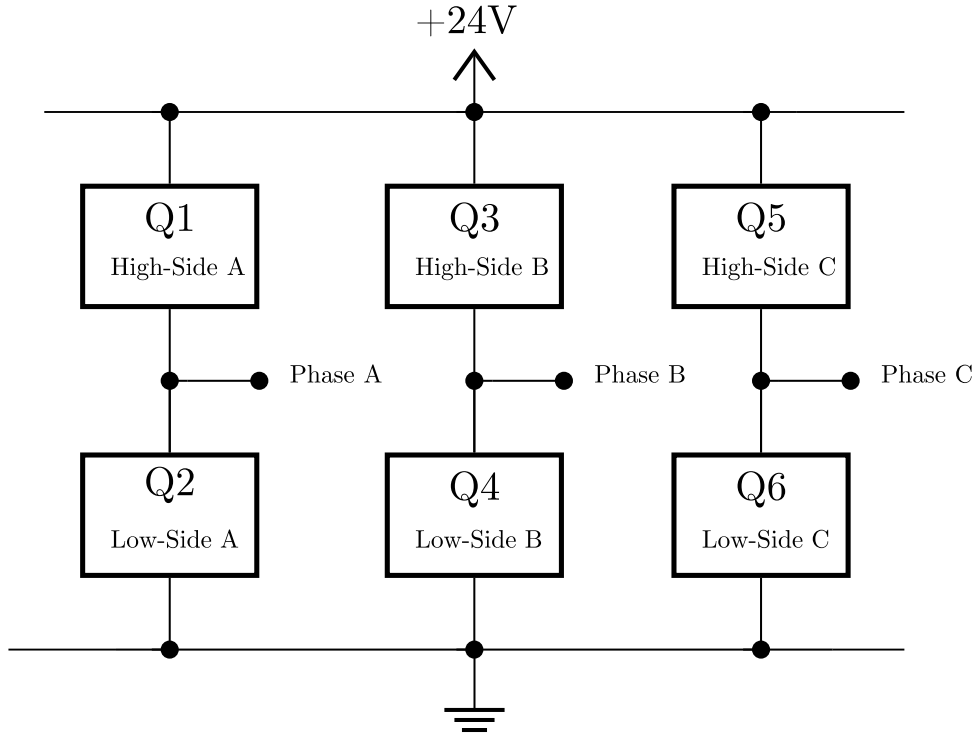


Figure 5: Simplified six-MOSFET three-phase bridge topology.

3.6 Shoot-Through Protection

In each inverter leg, the high-side and low-side MOSFETs must not turn ON at the same time. If both devices conduct simultaneously, the positive DC bus is connected directly to ground, producing a short-circuit condition known as shoot-through. This can cause a very large current spike and may damage the MOSFETs, gate drivers, or power supply.

To reduce this risk, the ESP32 MCPWM module is configured to generate complementary PWM signals with dead time. Dead time is a short delay inserted between turning one MOSFET OFF and turning the other MOSFET ON in the same leg. This ensures that one switch has enough time to turn OFF before the opposite switch starts conducting.

The detailed complementary waveform and the selected dead-time value are discussed later in the control and calculation sections.

4 Control Logic and ESP32 Waveforms

4.1 Software Flowchart

The ESP32 control algorithm starts by initializing the ADC input and configuring the MCPWM peripheral. The MCPWM outputs are assigned to the six inverter control signals and configured to generate complementary PWM pairs with dead time. During the main loop, the ESP32 reads the potentiometer value, maps it to the required output frequency, calculates the three SPWM duty cycles, and updates the PWM outputs continuously.

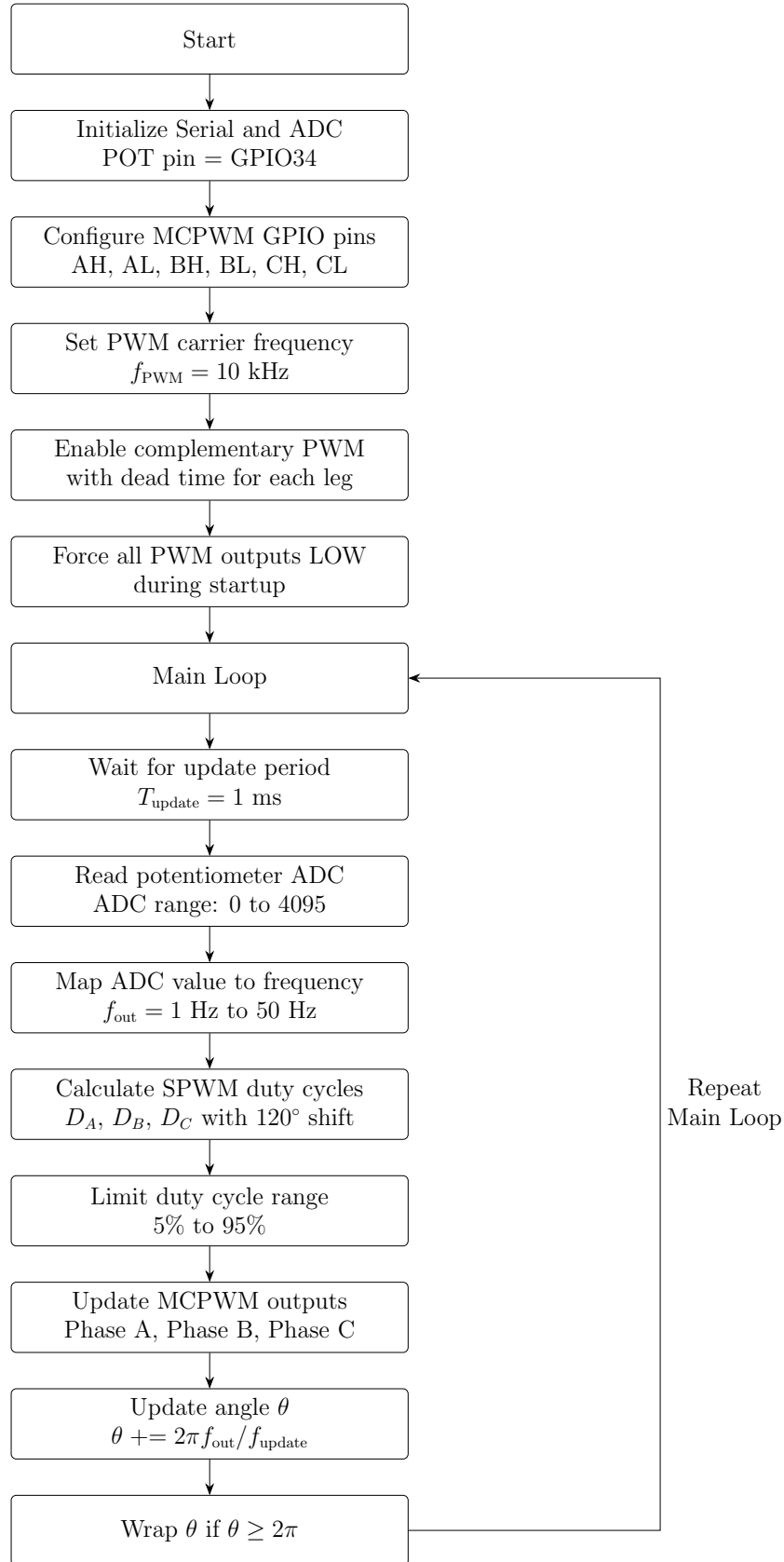


Figure 6: SPWM control flowchart for the ESP32-based three-phase inverter.

4.2 ESP32 MCPWM Module

The ESP32 MCPWM peripheral was used because it supports multiple PWM timers, paired PWM outputs, and hardware dead-time insertion. In this project, only the timer and operator blocks are used. The fault and capture blocks were not used because the inverter is controlled in open loop.

Three MCPWM timers are configured with the same carrier frequency:

$$f_{PWM} = 10 \text{ kHz} \quad (1)$$

Each timer controls one inverter phase. Output A is used for the high-side command, while output B is generated as the complementary low-side command with dead time.

Table 3: MCPWM timer and output assignment.

Timer	Phase	Output A	Output B
Timer 0	A	AH, GPIO25	AL, GPIO26
Timer 1	B	BH, GPIO27	BL, GPIO14
Timer 2	C	CH, GPIO12	CL, GPIO13

The timers are configured in up-counting, active-high mode. The duty cycle of output A is updated according to the SPWM equations, while output B is handled automatically by the complementary dead-time mode.

$$T_{PWM} = \frac{1}{10000} = 100 \mu s \quad (2)$$

The SPWM reference is updated at:

$$f_{update} = 1000 \text{ Hz}, \quad T_{update} = 1 \text{ ms} \quad (3)$$

Thus, the PWM carrier remains fixed at 10 kHz , while the sinusoidal duty reference is updated every 1 ms .

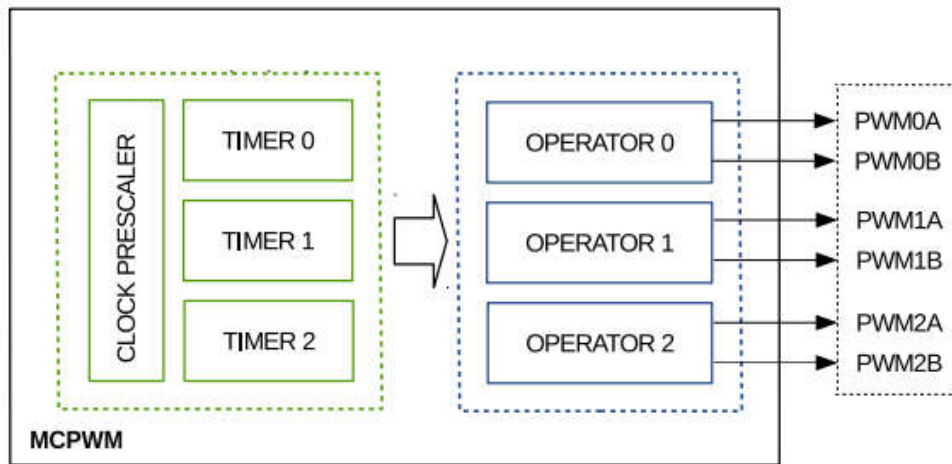


Figure 7: ESP32 MCPWM timer and operator blocks used for the three-phase inverter.

4.3 SPWM Duty-Cycle Generation

The inverter uses sinusoidal pulse-width modulation (SPWM) to generate the three-phase control signals. In this method, the duty cycle of each phase is varied according to a sinusoidal reference. The three references are shifted by 120° to produce a balanced three-phase output.

The duty cycles are calculated as:

$$D_A = 0.5 + 0.5m \sin(\theta) \quad (4)$$

$$D_B = 0.5 + 0.5m \sin\left(\theta - \frac{2\pi}{3}\right) \quad (5)$$

$$D_C = 0.5 + 0.5m \sin\left(\theta - \frac{4\pi}{3}\right) \quad (6)$$

where m is the modulation index. In this project:

$$m = 0.85 \quad (7)$$

The calculated duty cycle is limited between 5% and 95% to avoid extreme switching states and to help maintain bootstrap capacitor refreshing.

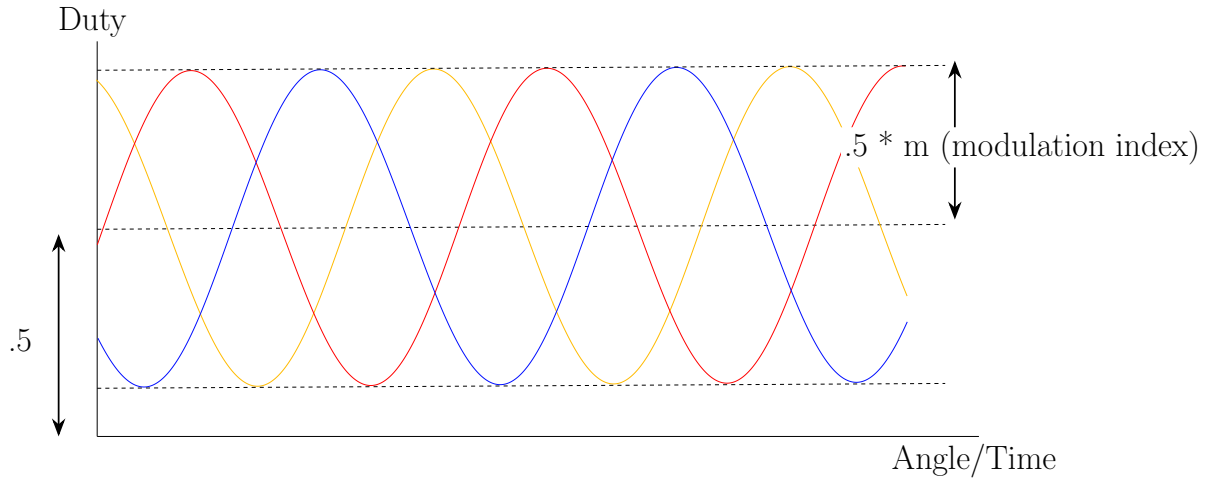


Figure 8: Three SPWM duty-cycle references shifted by 120° .

4.4 Six ESP32 PWM Command Signals

The ESP32 generates six PWM command signals for the three inverter legs. The high-side signals are A_H , B_H , and C_H , while the corresponding low-side signals are A_L , B_L , and C_L . Each low-side signal is generated as the complementary signal of its high-side pair, with dead time inserted between switching transitions.

The three phase commands are shifted by 120° , which allows the inverter to produce a balanced three-phase output. Figure ?? shows the intended six PWM command signals generated by the ESP32 before entering the IR2101 gate drivers.

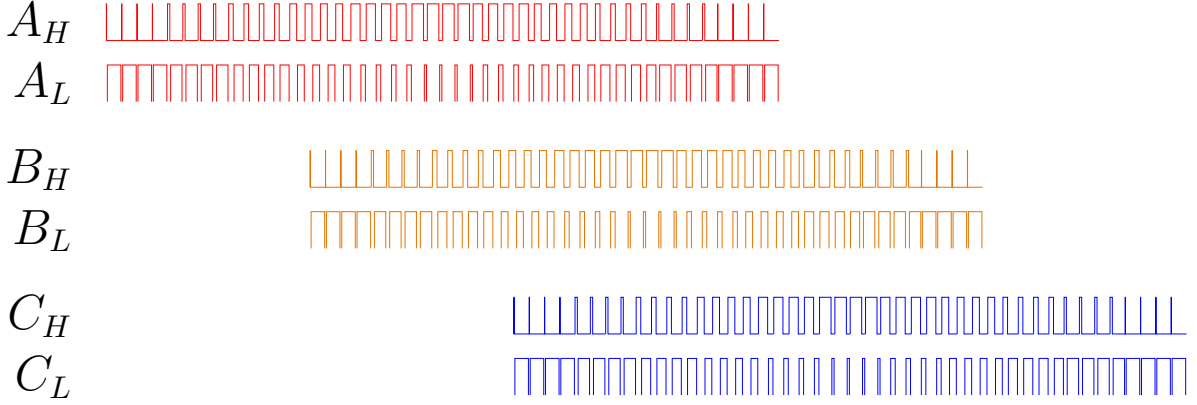


Figure 9: Three-phase complementary SPWM signals generated by the microcontroller. Each phase has high-side and low-side gate signals with dead time, while phases A, B, and C are shifted by 120° .

4.5 Complementary Switching and Dead Time

Each inverter phase leg contains one high-side MOSFET and one low-side MOSFET. These two switches must operate in a complementary manner, meaning that when one switch is ON, the other should be OFF. However, because MOSFETs and gate drivers have finite switching delays, the two signals must not change state at exactly the same time.

To reduce the risk of shoot-through, a short dead time is inserted between the turn-off of one MOSFET and the turn-on of the other MOSFET in the same leg. During this interval, both the high-side and low-side commands are LOW. Figure 10 shows a zoomed section of phase A, where A_H and A_L are complementary signals with dead time t_d inserted at each transition.

The same complementary switching method is applied to phases B and C. In the ESP32 code, this behavior is generated using the MCPWM dead-time feature, so each inverter leg receives a protected pair of PWM signals before being applied to the IR2101 gate-driver inputs. The detailed dead-time calculation and the selected programmed value are presented later in the **component selection section**.

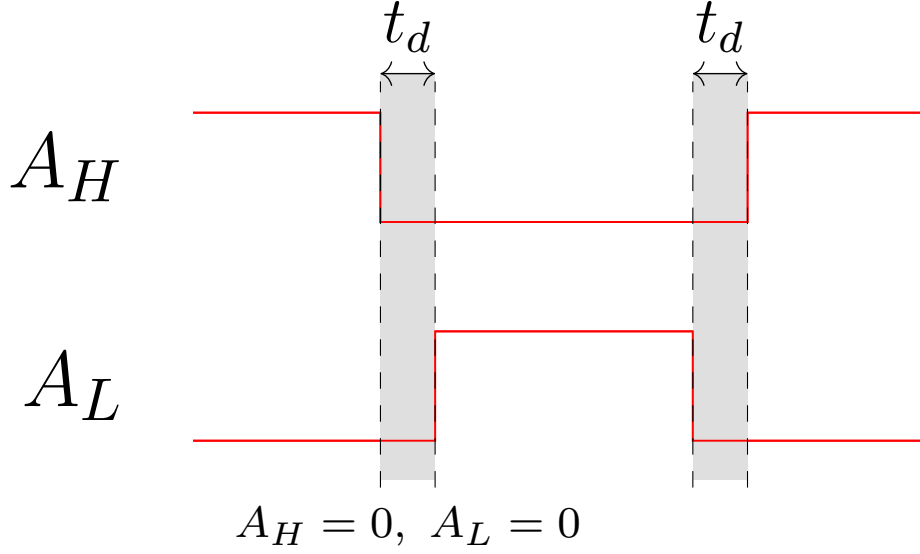


Figure 10: Zoomed section of one inverter phase showing complementary high-side and low-side signals with dead time.

4.6 Potentiometer-Based Frequency Control

The output frequency is controlled using a potentiometer connected to the ESP32 ADC input on GPIO34. The ESP32 reads the ADC value, which ranges from 0 to 4095, and maps it to the required fundamental output frequency range from 1 Hz to 50 Hz .

$$f_{out} = \text{map}(ADC, 0, 4095, 1, 50) \quad (8)$$

Changing the potentiometer position changes the SPWM reference frequency, which directly changes the inverter output frequency.

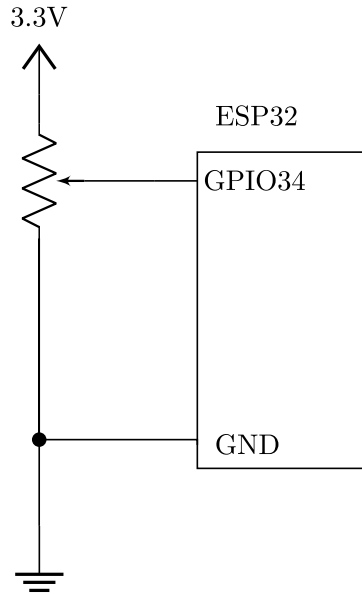


Figure 11: Potentiometer connection used for output frequency control.

4.7 Code Implementation and Availability

The ESP32 code configures the MCPWM module to generate three complementary PWM pairs with dead time. The potentiometer value is read through the ADC and mapped to the output frequency range. Then, three SPWM duty cycles shifted by 120° are calculated and continuously written to the MCPWM timers.

The complete code is also available in the project GitHub repository:

<https://github.com/Gha1th21/esp32-spwm-three-phase-inverter>

```
1 #include <Arduino.h>
2 #include <math.h>
3 #include "driver/mcpwm.h"
4
5 #define AH_PIN 25
6 #define AL_PIN 26
7
8 #define BH_PIN 27
9 #define BL_PIN 14
10
11 #define CH_PIN 12
12 #define CL_PIN 13
13
14 #define POT_PIN 34
15
16 #define PWM_FREQ_HZ 10000
17 #define ADC_MIN 0
18 #define ADC_MAX 4095
19
20 #define MIN_SINE_FREQ_HZ 1
21 #define MAX_SINE_FREQ_HZ 50
22
23 #define MOD_INDEX 0.85
24 #define UPDATE_RATE_HZ 1000
25
26 #define DEADTIME_TICKS 80
27
28 float theta = 0.0;
29 float sineFreq = 10.0;
30
31 unsigned long lastUpdateMicros = 0;
32
33 const unsigned long UPDATE_PERIOD_US = 1000000UL / UPDATE_RATE_HZ;
34
35 float calculateDuty(float angle)
36 {
37     float sineValue = sin(angle);
38
39     float duty = 0.5 + 0.5 * MOD_INDEX * sineValue;
40
41     if (duty > 0.95) duty = 0.95;
42     if (duty < 0.05) duty = 0.05;
43
44     return duty;
45 }
46
47 void setupMCPWM()
```

```

48 {
49     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0A, AH_PIN);
50     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0B, AL_PIN);
51
52     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM1A, BH_PIN);
53     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM1B, BL_PIN);
54
55     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM2A, CH_PIN);
56     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM2B, CL_PIN);
57
58     mcpwm_config_t pwm_config;
59     pwm_config.frequency = PWM_FREQ_HZ;
60     pwm_config.cmp_r_a = 0.0;
61     pwm_config.cmp_r_b = 0.0;
62     pwm_config.counter_mode = MCPWM_UP_COUNTER;
63     pwm_config.duty_mode = MCPWM_DUTY_MODE_0;
64
65     mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &pwm_config);
66     mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_1, &pwm_config);
67     mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_2, &pwm_config);
68
69     mcpwm_deadtime_enable(
70         MCPWM_UNIT_0,
71         MCPWM_TIMER_0,
72         MCPWM_ACTIVE_HIGH_COMPLIMENT_MODE,
73         DEADTIME_TICKS,
74         DEADTIME_TICKS
75     );
76
77     mcpwm_deadtime_enable(
78         MCPWM_UNIT_0,
79         MCPWM_TIMER_1,
80         MCPWM_ACTIVE_HIGH_COMPLIMENT_MODE,
81         DEADTIME_TICKS,
82         DEADTIME_TICKS
83     );
84
85     mcpwm_deadtime_enable(
86         MCPWM_UNIT_0,
87         MCPWM_TIMER_2,
88         MCPWM_ACTIVE_HIGH_COMPLIMENT_MODE,
89         DEADTIME_TICKS,
90         DEADTIME_TICKS
91     );
92 }
93
94 void writePhasePWM(mcpwm_timer_t timer, float duty)
95 {
96     float dutyPercent = duty * 100.0;
97
98     if (dutyPercent > 95.0) dutyPercent = 95.0;
99     if (dutyPercent < 5.0) dutyPercent = 5.0;
100
101     mcpwm_set_duty(MCPWM_UNIT_0, timer, MCPWM_OPR_A, dutyPercent);
102     mcpwm_set_duty_type(MCPWM_UNIT_0, timer, MCPWM_OPR_A,
103         MCPWM_DUTY_MODE_0);
104 }

```



```

105 void allOff()
106 {
107     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_0, MCPWM_OPR_A);
108     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_0, MCPWM_OPR_B);
109
110     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_1, MCPWM_OPR_A);
111     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_1, MCPWM_OPR_B);
112
113     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_2, MCPWM_OPR_A);
114     mcpwm_set_signal_low(MCPWM_UNIT_0, MCPWM_TIMER_2, MCPWM_OPR_B);
115 }
116
117 void setup()
118 {
119     Serial.begin(115200);
120     delay(1000);
121
122     pinMode(POT_PIN, INPUT);
123
124     setupMCPWM();
125     allOff();
126
127     delay(1000);
128
129     Serial.println("ESP32_3-Phase_SPWM_with_MCPWM_Dead-Time_Started");
130 }
131
132 void loop()
133 {
134     unsigned long now = micros();
135
136     if (now - lastUpdateMicros >= UPDATE_PERIOD_US)
137     {
138         lastUpdateMicros = now;
139
140         int adcValue = analogRead(POT_PIN);
141
142         sineFreq = map(adcValue, ADC_MIN, ADC_MAX,
143                       MIN_SINE_FREQ_HZ, MAX_SINE_FREQ_HZ);
144
145         float dutyA = calculateDuty(theta);
146         float dutyB = calculateDuty(theta - 2.0 * PI / 3.0);
147         float dutyC = calculateDuty(theta - 4.0 * PI / 3.0);
148
149         writePhasePWM(MCPWM_TIMER_0, dutyA);
150         writePhasePWM(MCPWM_TIMER_1, dutyB);
151         writePhasePWM(MCPWM_TIMER_2, dutyC);
152
153         theta += 2.0 * PI * sineFreq / UPDATE_RATE_HZ;
154
155         if (theta >= 2.0 * PI)
156         {
157             theta -= 2.0 * PI;
158         }
159     }
160 }

```

Listing 1: ESP32 code for three-phase SPWM generation with MCPWM dead time.

5 Component Selection and Calculations

5.1 Gate Resistor Selection

The MOSFET gate acts like a small capacitor, so a gate resistor is used to limit the current from the IR2101 driver.

From the IR2101 output-current curves at $V_{CC} = 12\text{ V}$, shown in Figure 12, the driver source and sink currents are approximately:

$$I_{O+} \approx 150\text{ mA}, \quad I_{O-} \approx 300\text{ mA} \quad (9)$$

Using $R = V/I$ with a 12 V gate-drive supply:

$$R_{g,on} = \frac{12}{0.15} = 80\ \Omega \quad (10)$$

$$R_{g,off} = \frac{12}{0.3} = 40\ \Omega \quad (11)$$

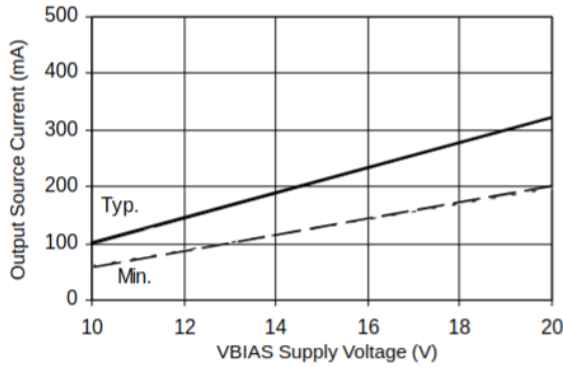
Therefore, a practical gate-resistance range is approximately:

$$40\ \Omega \leq R_g \leq 80\ \Omega \quad (12)$$

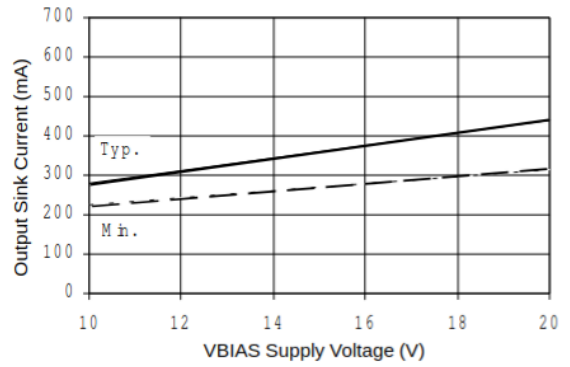
A value of $75\ \Omega$ was selected, giving an approximate peak gate current of:

$$I_g = \frac{12}{75} = 0.16\text{ A} = 160\text{ mA} \quad (13)$$

This value is close to the IR2101 source-current capability while still limiting the gate current. Therefore, $75\ \Omega$ provides a suitable compromise between driver protection, reduced switching noise, and sufficient switching speed.



(a) Output source current versus V_{BIAS} .



(b) Output sink current versus V_{BIAS} .

Figure 12: IR2101 output source and sink current curves used for gate resistor selection. At $V_{BIAS} = 12\text{ V}$, the approximate source and sink current capabilities were used to estimate a suitable gate resistance range.

5.2 Bootstrap Capacitor Selection

The bootstrap capacitor supplies the floating high-side section of the IR2101 driver when the high-side MOSFET is ON. It must provide enough charge for the MOSFET gate, the IR2101 high-side supply current, level shifting, and leakage currents without allowing the bootstrap voltage to drop below the required value.

The required bootstrap capacitance was calculated using:

$$C_{boot} \geq \frac{2 \left(2Q_g + \frac{I_{QBS(max)}}{f} + Q_{LS} + \frac{I_{CBS}}{f} \right)}{\Delta V} \quad (14)$$

where the factor of 2 is used as a safety factor. The values of Q_g , $I_{QBS(max)}$, and $V_{BS(min)}$ were taken from the IRFZ44 and IR2101 datasheets, while the level-shifting charge and leakage current were chosen as typical design values.

The values used were:

$$Q_g = 62 \text{ nC}, \quad I_{QBS(max)} = 55 \text{ }\mu\text{A}, \quad Q_{LS} = 5 \text{ nC}, \quad I_{CBS} = 2 \text{ }\mu\text{A} \quad (15)$$

The worst-case refresh frequency was taken as:

$$f = 1 \text{ kHz} \quad (16)$$

The allowable bootstrap voltage drop is:

$$\Delta V = V_{CC} - V_f - V_{LS} - V_{BS(min)} \quad (17)$$

$$\Delta V = 12 - 0.7 - 0.2 - 10 = 1.1 \text{ V} \quad (18)$$

Substituting the values:

$$Q_{total} = 2(62 \text{ nC}) + 55 \text{ nC} + 5 \text{ nC} + 2 \text{ nC} = 186 \text{ nC} \quad (19)$$

After applying the safety factor:

$$Q_{safe} = 2(186 \text{ nC}) = 372 \text{ nC} \quad (20)$$

Therefore:

$$C_{boot} \geq \frac{372 \text{ nC}}{1.1 \text{ V}} = 338 \text{ nF} \quad (21)$$

The selected standard value was:

$$C_{boot} = 470 \text{ nF} \quad (22)$$

This value is higher than the calculated minimum, so it provides a safety margin and helps keep the high-side supply voltage above the required minimum during operation.

5.3 Dead-Time Selection

Dead time is required to prevent the high-side and low-side MOSFETs in the same inverter leg from turning ON at the same time. Based on the IR2101 timing parameters, the minimum required dead time was estimated using:

$$T_{dead} = T_{d(off)} + T_f - T_{d(on)} + MT \quad (23)$$

where $T_{d(off)}$ is the turn-off delay time, T_f is the fall time, $T_{d(on)}$ is the turn-on delay time, and MT is the delay matching between the high-side and low-side channels.

From the IR2101 datasheet timing values shown in Figure 13:

$$T_{d(off)} = 50 \text{ ns}, \quad T_f = 40 \text{ ns}, \quad T_{d(on)} = 26 \text{ ns}, \quad MT = 50 \text{ ns} \quad (24)$$

Therefore:

$$T_{dead} = 50 + 40 - 26 + 50 = 114 \text{ ns} \quad (25)$$

The calculated value represents the theoretical minimum. In the ESP32 MCPWM library, dead time is set using timer ticks rather than directly in seconds. Therefore, the practical dead time was calibrated using the oscilloscope. After calibration, a dead time of approximately:

$$T_{dead,used} = 5 \mu s \quad (26)$$

was used in the PWM configuration. This value is larger than the calculated minimum and provides a safety margin against propagation-delay variation, MOSFET switching delay, and parasitic effects.

$t_{d(on)}$	Turn-on delay time	$V_{DD} = 30 \text{ V}; I_D = 25 \text{ A};$	-	18	26	ns
t_r	Turn-on rise time	$V_{GS} = 10 \text{ V}; R_G = 10 \Omega$	-	50	75	ns
$t_{d(off)}$	Turn-off delay time	Resistive load	-	40	50	ns
t_f	Turn-off fall time		-	30	40	ns

(a) MOSFET switching-time parameters used in the dead-time calculation.

t_{off}	Turn-off propagation delay	—	150	220	ns	$V_S = 600 \text{ V}$
t_r	Turn-on rise time	—	100	170		
t_f	Turn-off fall time	—	50	90		
MT	Delay matching, HS & LS turn-on/off	—	—	50		

(b) IR2101 high-side and low-side delay matching parameter.

Figure 13: Timing parameters used for estimating the minimum required dead time.

5.4 Wire Selection

Since the inverter may be used to drive motor loads, the power-stage wiring must be able to carry relatively high current safely. Therefore, wires with a suitable diameter were selected for the DC bus and the three-phase output connections. This reduces voltage drop, limits wire heating, and improves the reliability of the inverter during operation.

5.5 Heatsink Selection and Thermal Check

The MOSFET power dissipation was first estimated using the conduction-loss equation:

$$P_{dis} = I^2 R_{DS(on)} \quad (27)$$

For the expected operating current, the drain current was assumed to be:

$$I = 10 \text{ A} \quad (28)$$

Using $R_{DS(on)} = 22 \text{ m}\Omega$ from the MOSFET datasheet at $V_{GS} = 10 \text{ V}$:

$$P_{dis} = 10^2(0.022) = 2.2 \text{ W} \quad (29)$$

The required heatsink thermal resistance was calculated from the junction-to-ambient thermal path. The thermal path includes the MOSFET junction-to-case resistance, the case-to-sink interface, and the heatsink-to-ambient resistance. In this simplified calculation, the case-to-sink resistance was neglected because **thermal paste was used**.

$$R_{\theta SA} \leq \frac{T_J - T_A}{P_{dis}} - R_{\theta JC} \quad (30)$$

Using $T_J = 125^\circ\text{C}$, $T_A = 25^\circ\text{C}$, and $R_{\theta JC} = 1.4^\circ\text{C/W}$:

$$R_{\theta SA} \leq 44^\circ\text{C/W} \quad (31)$$

Therefore, the selected heatsink must have a thermal resistance lower than approximately 44°C/W . A heatsink with:

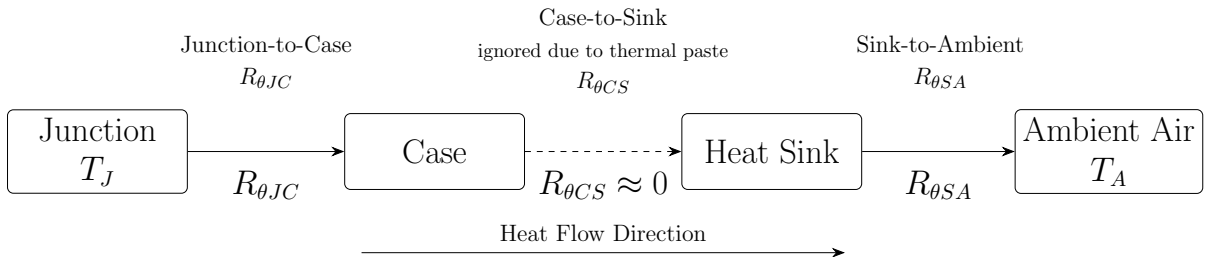
$$R_{\theta SA} = 20^\circ\text{C/W} \quad (32)$$

was selected, which provides a good safety margin. The expected junction temperature is:

$$T_J = T_A + P_{dis}(R_{\theta JC} + R_{\theta SA}) = 25 + 2.2(1.4 + 20) \quad (33)$$

$$T_J \approx 72^\circ\text{C} \quad (34)$$

Since the estimated junction temperature is below the selected limit of 125°C , the heatsink is suitable for the assumed operating current and ambient temperature.



Simplified thermal path used in the design:

$$T_J = T_A + P_{dis}(R_{\theta JC} + R_{\theta SA})$$

Figure 14: Simplified MOSFET thermal path for heat-sink selection.

6 Experimental Results and Oscilloscope Measurements

6.1 ESP32 PWM Output Measurements

Figure 15 shows the measured complementary PWM signals generated by the ESP32 for one inverter phase. The two signals are opposite to each other, confirming that the MCPWM module is producing the required high-side and low-side command pair. The measured waveform also verifies the switching behavior before the signals are applied to the IR2101 gate driver.

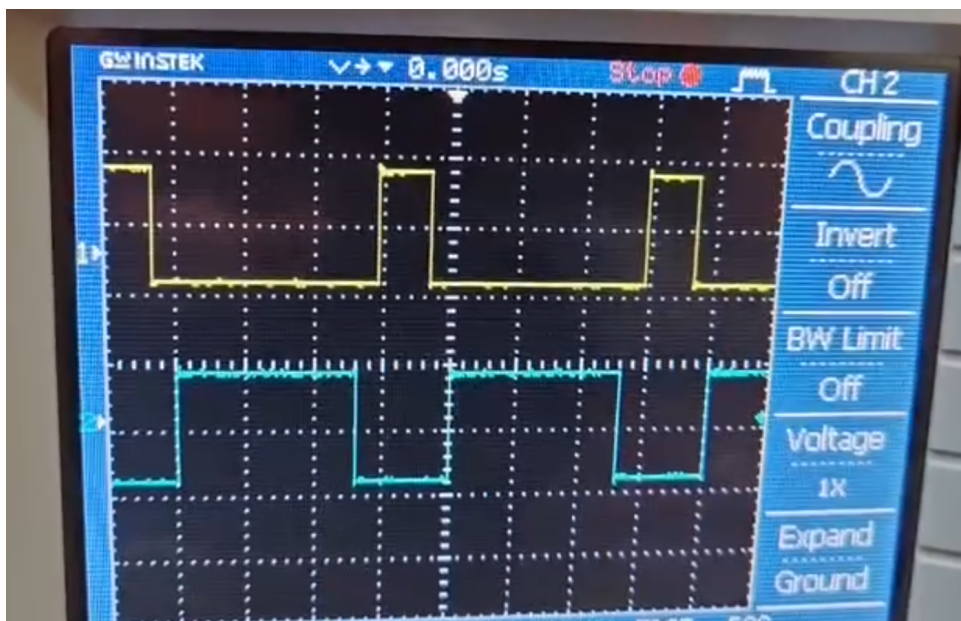
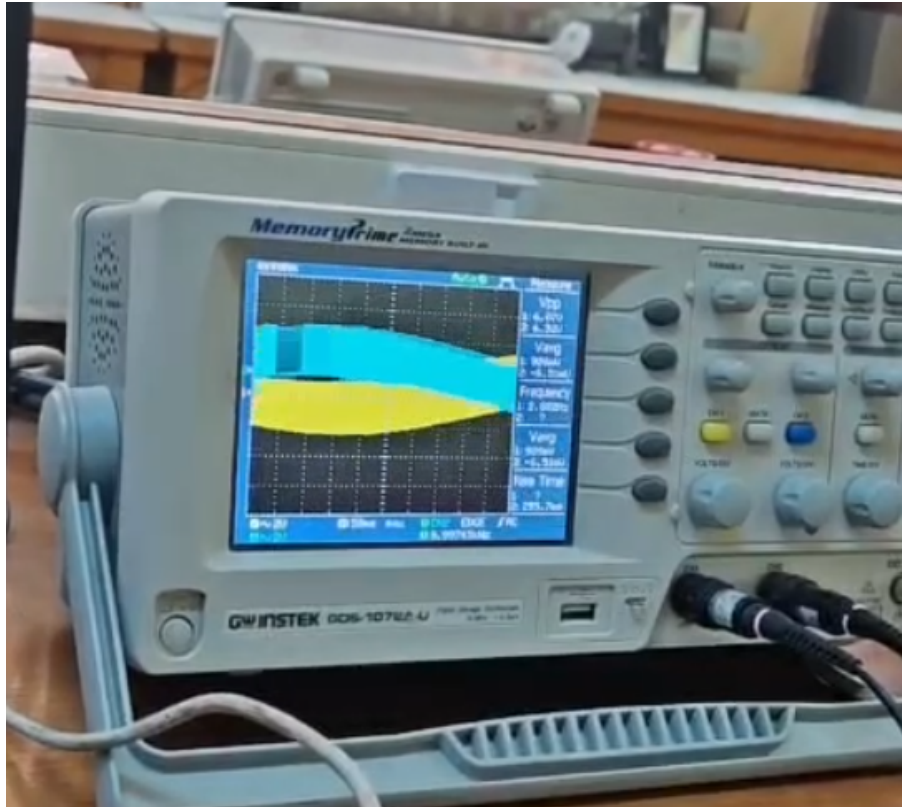


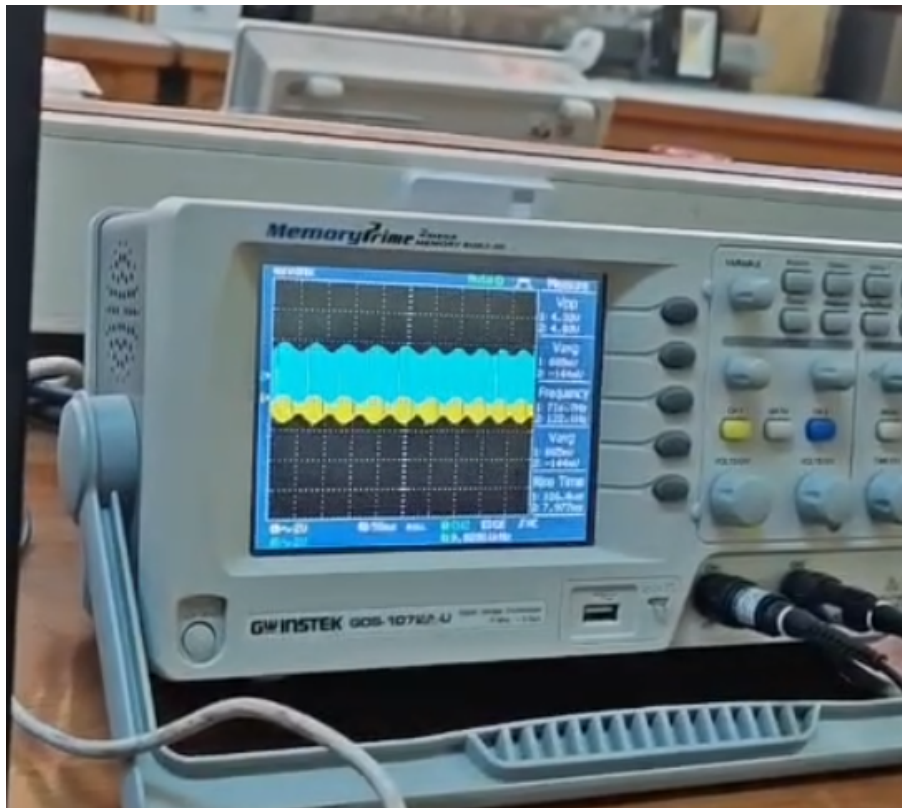
Figure 15: Measured ESP32 complementary PWM output signals for one inverter phase.

6.2 Three-Phase Inverter Output Measurements

Figure 16 shows the measured inverter output waveforms at two different output frequencies. The change in waveform period confirms that the potentiometer successfully changes the SPWM fundamental frequency. The measured outputs also show the expected switching behavior of the three-phase inverter.



(a) Low output frequency.



(b) High output frequency.

Figure 16: Measured inverter output waveforms at low and high output frequencies.

6.3 Measurement Discussion

The oscilloscope measurements confirm that the ESP32 generated complementary PWM signals and that the inverter output frequency changed according to the potentiometer position. At lower frequency, the output waveform period is longer, while at higher frequency the period becomes shorter. The measured results agree with the expected SPWM-based inverter operation.

7 Discussion and Conclusion

The three-phase inverter was successfully designed and implemented using an ESP32, three IR2101 gate drivers, and six IRFZ44 MOSFETs. The ESP32 generated SPWM signals with complementary outputs and dead time using the MCPWM module, while the potentiometer allowed the output frequency to be varied from 1 Hz to 50 Hz .

The IR2101 drivers provided the required gate-drive interface between the low-voltage ESP32 signals and the MOSFET bridge. The bootstrap capacitors enabled high-side MOSFET operation, and the gate resistors, dead time, heatsinks, and wire sizes were selected to improve safe and reliable operation.

The oscilloscope measurements confirmed the expected complementary PWM behavior and showed that the inverter output frequency changed when the potentiometer was adjusted. For future improvement, the design could be implemented on a PCB with better grounding, stronger decoupling, current sensing, and overcurrent protection.

8 References

1. Infineon/International Rectifier, IR2101 High and Low Side Driver Datasheet.
2. International Rectifier, IRFZ44 Power MOSFET Datasheet.
3. Espressif Systems, ESP32 MCPWM Documentation.
4. CUI Devices, *Thermal Management: The Complete Guide*.
5. Labcenter Electronics, Proteus Design Suite Documentation.